



DOCUMENTO 2: HOJA DE AYUDA (CHEAT SHEET)



HOJA DE AYUDA: Programación Asíncrona y Vue 3

Taller Semana 8 - Motor Contable ERP Ten este documento a la mano mientras realizas los ejercicios.

1. El Patrón de Oro: `try / catch / finally`

Siempre que hagas una petición HTTP, usa esta estructura. Garantiza que tu aplicación no se congele.

```
const cargando = ref(false);
const error = ref(null);
const datos = ref([]);

async function cargarDatos() {
  cargando.value = true; // 1. Mostrar spinner
  error.value = null;    // 2. Limpiar errores previos

  try {
    // 3. Petición asíncrona
    const respuesta = await api.get('/movimientos');
    datos.value = respuesta.data.datos;

  } catch (err) {
    // 4. Si falla (red caída, error 500, etc.)
    console.error('Error:', err);
    error.value = 'No se pudieron cargar los datos. Intenta más tarde.';

  } finally {
    // 5. SIEMPRE se ejecuta. Oculta el spinner.
    cargando.value = false;
  }
}
```

2. Axios vs Fetch API

¿Por qué usamos Axios en este taller?

- **Fetch (Nativo):** Requiere llamar a `.json()` manualmente y NO lanza error si el servidor responde con un 404 o 500.
- **Axios (Librería):** Parsea el JSON automáticamente y lanza la excepción al `catch` si el código de estado no es 2xx.

Configuración Centralizada (`src/services/erpApi.js`):

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:3000/api', // Ya no repites esta URL
  timeout: 10000, // 10 segundos máximo
  headers: { 'Content-Type': 'application/json' }
});

export const movimientoService = {
  getAll: () => api.get('/movimientos'),
  create: (datos) => api.post('/movimientos', datos)
};

export default api;
```

3. Reactividad y Ciclo de Vida en Vue 3

ref() vs reactive()

- Usa **ref()** para casi todo (primitivos, arrays, objetos). **Recuerda usar .value** dentro del `<script setup>`.
- Usa **reactive()** solo para objetos complejos donde no quieras escribir **.value**.

onMounted()

Es el hook ideal para hacer tu primera petición HTTP. Se ejecuta justo cuando el componente ya está dibujado en la pantalla.

```
import { onMounted } from 'vue';

onMounted(() => {
  cargarMovimientos(); // Llama a tu función async aquí
});
```

4. Propiedades Computadas (computed)

Ideales para el **Motor Contable**. Se recalculan automáticamente *solo* cuando los datos que leen cambian.

```
import { computed } from 'vue';

const movimientos = ref([]);

const totalIngresos = computed(() => {
  // Manejo de array vacío para evitar NaN
  if (!movimientos.value.length) return 0;

  return movimientos.value
```

```
.filter(m => m.tipo === 'Ingreso')
.reduce((suma, m) => suma + m.monto, 0);
});

const saldo = computed(() => totalIngresos.value - totalEgresos.value);
```

5. Manejo de Errores HTTP con Axios

Axios estructura los errores de una forma muy útil dentro del bloque `catch`:

```
catch (err) {
  if (err.response) {
    // El servidor respondió con un código de error (400, 404, 500)
    console.log(err.response.status);

    if (err.response.status === 400) {
      // Muestra el mensaje exacto que envió tu backend Express
      error.value = err.response.data.mensaje;
    }
  } else if (err.request) {
    // La petición se envió pero no hubo respuesta (Servidor caído / CORS)
    error.value = 'El servidor no responde. Verifica que esté encendido.';
  } else {
    // Error de configuración
    error.value = 'Error inesperado en la aplicación.';
  }
}
```

6. Errores Comunes y Debugging 🐛

Error en Consola	Causa Probable	Solución
Cannot read properties of undefined	Olvidaste el <code>.value</code> en un <code>ref</code> o la API no devolvió la estructura esperada.	Revisa <code>console.log(respuesta)</code> para ver la estructura real.
await is only valid in async functions	Usaste <code>await</code> en una función normal.	Agrega la palabra <code>async</code> antes de <code>function nombre()</code> .
CORS error en el navegador	El backend no permite peticiones desde el puerto 5173.	Asegúrate de que <code>mi-servidor</code> tenga <code>app.use(cors())</code> .
El spinner nunca se oculta	Olvidaste el bloque <code>finally</code> o hubo un error antes de <code>cargando = false</code> .	Revisa la estructura <code>try/catch/finally</code> .

💡 **Tip de Debugging:** Si algo falla, abre las **Herramientas de Desarrollador (F12)** -> Pestaña **Network (Red)**. Filtra por **Fetch/XHR**. Ahí verás exactamente qué le enviaste al servidor y qué te respondió (Status

Code y Body).